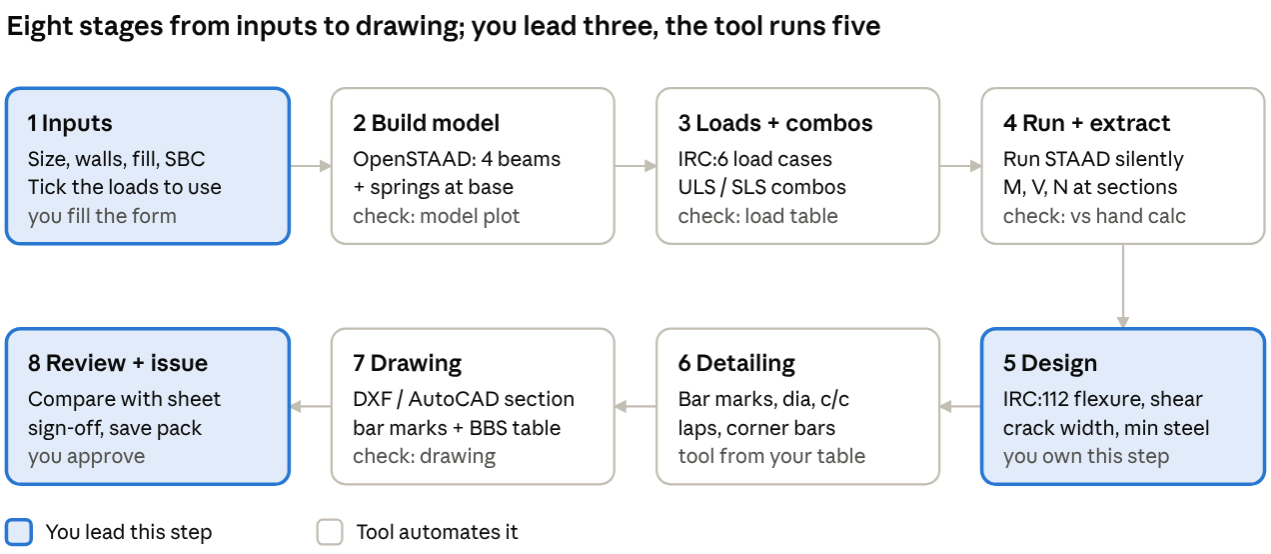
Box Culvert Automation – Project Plan

Sep 28, 2026 · @Muhammed Fasil

At a glance

Goal: type a culvert's size and loads once, and get a STAAD.Pro model, extracted forces, and a detailed AutoCAD section out the other end. Python drives STAAD.Pro through its **OpenSTAAD** API (this replaces your VBA extraction) and writes the drawing as DXF or through AutoCAD's COM API.



project pipeline · 8 stages

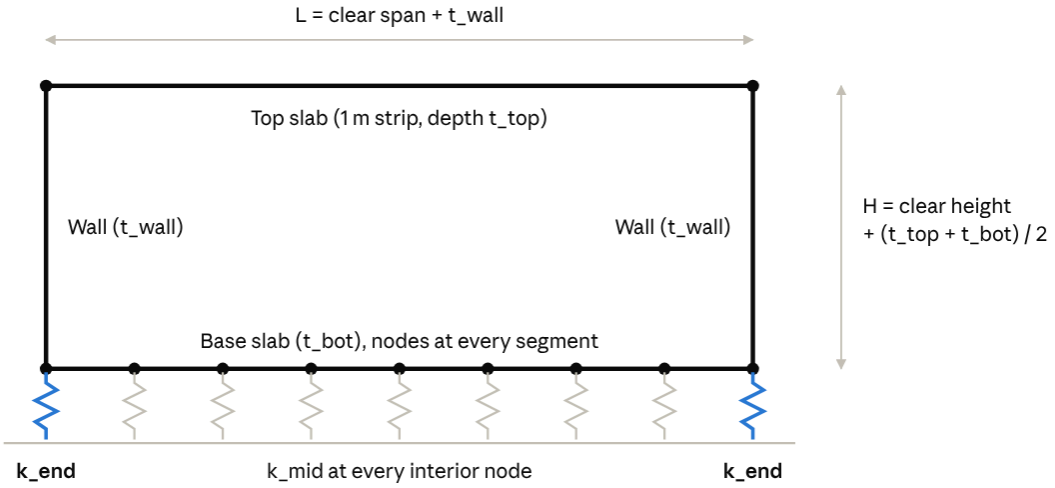
The tool does the repetitive work (modelling, loading, extraction, drafting). You keep the engineering decisions: the inputs, the design, and the final check against your own sheet.

Scope for version 1: single-cell RCC box, 1 m wide strip, line (beam) model on Winkler springs, IRC:6-2017 loads, IRC:112 design, one standard cross-section drawing.

Stage 1 – Geometry and model

The culvert becomes a closed 2D frame drawn on the **centrelines** of the slabs and walls, 1 m deep into the page. Four members (top, two walls, base); the base is split into n pieces so every piece-end gets a spring.

Centreline frame on springs: 4 members, base slab split into n segments



Soil modelled as vertical Winkler springs (Stage 2)

STAAD line model · 1 m strip

Input	Symbol	Unit	Used for
Clear span (inside width)	B	m	L = B + t_wall
Clear height (inside)	D	m	H = D + (t_top + t_bot)/2
Top slab thickness	t_top	m	section depth, member 1
Wall thickness	t_wall	m	section depth, members 2–3
Base slab thickness	t_bot	m	section depth, member 4
Concrete grade	M__	–	E (Ecm, IRC:112 Table 6.5), self-weight 25 kN/m³ (IRC:6 cl. 203)
Base segments	n	nos	spring spacing = L / n

What the tool does: computes node coordinates, creates beams, assigns rectangular sections (depth = thickness, width = 1.0 m) and concrete, then shows you a plot to confirm before loading.

For a 4 × 4 or 4 × 5 box, n = 10 to 20 on the base and 4 to 8 on each wall is plenty; walls are split too so earth-pressure results can be read at intermediate points.

> **Important note (16 Oct): base slab layout, overrides the segment rule above**
>
> - The base slab runs to the **outer face** of the walls: it extends t_wall/2 past each wall centreline. The wall–slab junction nodes stay on the centreline.
> - Input: number of intermediate spring nodes, equally spaced between the two junction nodes.
> - Example: centreline span 5.0 m, t_wall 0.4 m, 9 intermediate nodes → effective base length 5.4 m. Nodes at **0, 0.2, 0.7 … 4.7 (9 nodes at 0.5 m), 5.2, 5.4**.
> - Concrete density 25 kN/m³: set it the same way the sample STAAD file does.
> - The twin-cell Excel and STAAD files are reference for the *method* only; the tool stays single cell.
> - The Stage 1 script needs this change (two short end members on the base slab).

Stage 2 – Soil springs

Your "40 × FOS × SBC" rule is Bowles' approximation for the modulus of subgrade reaction, taken from *Foundation Analysis and Design* (summary). It assumes the ultimate soil pressure occurs at about 25 mm settlement, so 40 is on the conservative side.

$$k_s = 40 \cdot FOS \cdot SBC \quad [\mathrm{kN/m^3}], \; SBC \text{ in kPa}$$

$$K_{node} = k_s \cdot (\text{tributary length}) \cdot 1.0\,\mathrm{m} \quad [\mathrm{kN/m}]$$

Spring	Tributary length	Value
Interior node (k_mid)	s = L / n	ks × s × 1.0
End node (k_end), geometric	s/2 + t_wall/2 (the slab runs past the centreline to the outer face)	ks × (s/2 + t_wall/2) × 1.0
End node (k_end), Bowles option	as above, then doubled to mimic stiffer soil under edges	2 × geometric value

The end spring differs from the middle ones for a geometric reason: the end node carries half a segment plus the half-wall overhang. Which end rule your sheet uses is an open question below.

Tool steps: read SBC and FOS → compute ks → compute k_mid and k_end → create two spring supports in STAAD (vertical spring KFY, moment released) → assign to base nodes → after analysis, check base pressure = reaction ÷ tributary area ⩽ SBC under SLS.

Decisions to make:

- Horizontal restraint: fix FX at one node, or add small horizontal springs at all base nodes.
- Compression-only springs (lift-off at edges) need STAAD's non-linear iterative analysis; linear springs are simpler for version 1.

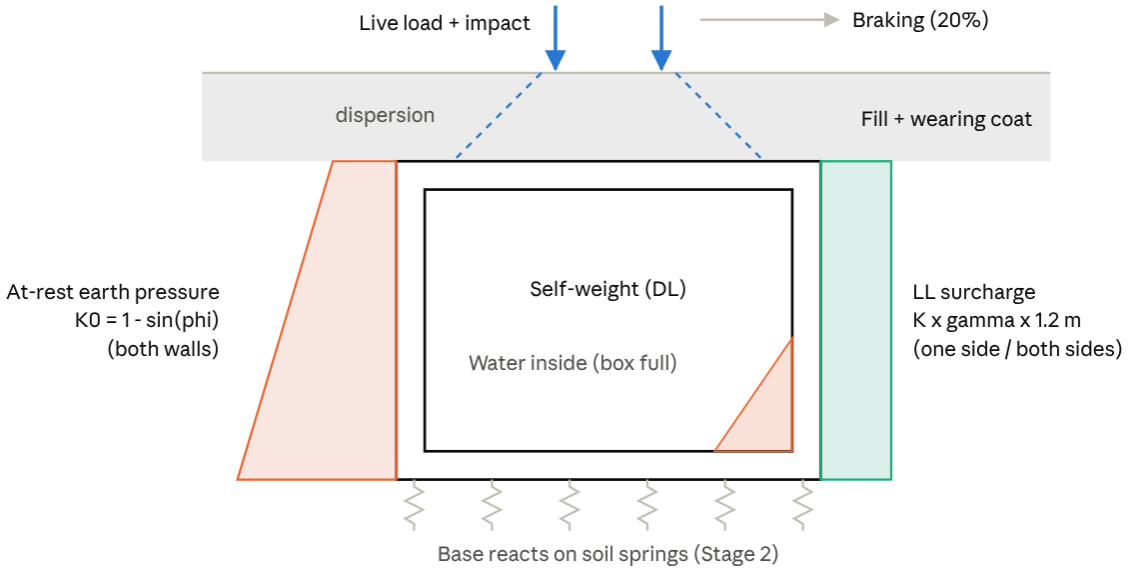
> **Important note (16 Oct): only three supports in STAAD, no more**
>
> - K = ks × tributary length × 1 m, with ks = 40 × FOS × SBC. Tributary length = half the gap on each side of the node.
> - Compression-only springs stay (decided earlier).
> - Check: tributary lengths add up to the effective base length (5.4 m in the example).
> - The Stage 2 script will be rewritten for this layout and the shared-module structure.
> - **To confirm:** end supports "release X and Z" = FX and FZ released, everything else fixed? And "FY only" on the other two = all other directions fixed? This sets the horizontal restraint and replaces the earlier "fix FX at one node" choice.

Support	Nodes	Spring and releases	Tributary length (example)
End	2 outer-face nodes (0, 5.4)	KFY; FX and FZ released	0.10 m
Wall junction	2 junction nodes (0.2, 5.2)	KFY only	0.10 + 0.25 = 0.35 m
Intermediate	all n intermediate nodes	KFY only	0.50 m

Stage 3 – Loads (IRC:6-2017)

IRC:6-2017 is still the current loading code; IRC has kept amending it (Amendment No. 9 was notified in March 2026), so load the latest amendments into the checklist (IRC amendment list). Each row below becomes one tick-box line in the tool.

Loads on the box: vertical from above, lateral from both sides



load picture · single-cell box

#	Load	Rule	IRC:6-2017 clause
1	Self-weight (DL)	RCC 2.5 t/m³	203
2	Wearing coat / SIDL	asphalt 2.2 t/m³	203
3	Earth fill on top	compacted earth 2.0 t/m³ × fill depth	203
4	Earth pressure on walls	at rest, K0 = 1 − sin φ (closed box named as the example)	214.1.3
5	Live-load surcharge	K × γ × 1.2 m equivalent height	214.1.1.3
6	Vehicle live load	Class 70R, Class A, SV if the corridor needs it; lanes by carriageway width	204.1, Table 6, 204.5
7	Impact	Class A: 4.5/(6+L); 70R tracked 25% up to 5 m, linearly to 10% at 9 m; 70R wheeled 25% below 9 m span; halved when fill ≥ 0.6 m; none for SV	208.2, 208.3, 208.6, 204.5.4
8	Braking	20% of first train, 1.2 m above road, no impact	211.2, 211.3, 211.7
9	Water	inside (box full); hydrostatic behind walls if no drainage	214.3, 214.4

#	Load	Rule	IRC:6-2017 clause
10	Temperature, shrinkage	shrinkage strain 2×10^{-4}	215, 216.3
11	Seismic	not needed for culverts up to 10 m span	218.1.1

Source: IRC:6-2017 text.

Cases the tool must build (the classic box set): box empty vs box full; surcharge on one side vs both; live load placed for maximum sagging at mid top slab, maximum hogging at the corners, and maximum shear near the wall face.

Live load on a 2D strip: the tool places the wheel or track loads, spreads them through the fill, and divides by the effective width to get kN per metre strip. That dispersion rule is an open question — we will lift it from your design sheet.

> **Important note (16 Oct): load cases for this version**
>
> 1. **Dead load:** STAAD SELFWEIGHT command (density 25 kN/m³).
> 2. **SIDL:** calculate both, load the **max** into STAAD; the calc sheet shows both.
>   - (a) SIDL / wearing coat / fill over the full 1 m strip: Σ(h × γ) × 1 m → UDL.
>   - (b) Crash barrier width b (e.g. 0.5 m): barrier area × γ, plus SIDL / fill pressure × (1 − b) → UDL.
> 3. **Earth pressure:** Excel method, on both walls. The Excel is correct; the sample STAAD used conservative values (5 / 49 vs 4.59 / 48.34 kN/m²). The tool loads the calculated values.
> 4. **LL surcharge, both sides.**
> 5. **LL surcharge, one side.**
> 6. **Braking:** positions as in the sample STAAD, values as in the Excel.
> 7. **Water inside the cell (new, not in the Excel or STAAD):** triangular on both walls, 0 at height H above the top of the base slab, rising to γw·H at the top of the base slab (t_bot/2 above the base centreline).
>
> - **Not in this version:** seismic (the tool flags spans above 10 m); temperature (handled separately in the next version).
> - **Vehicle live load: next session.** Five cases: IRC 70R tracked (70T); 70R wheeled bogie (40T); 70R wheeled; Class A single lane; Class A two lane. Each load is dispersed through fill / SIDL / wearing coat, b_eff is taken from IRC:112-2020 Annex B-3, and the result goes into STAAD as a line load on the 1 m strip. You will share the STAAD model and matching Excel sheet for this. Explain first, implement once we agree.
> - **Output:** the load calculation comes out in your EDRC format, not the twin-cell Excel's layout.
> - **To confirm:** does the water case also put γw·H downward on the base slab?

Stage 4 – Load combinations (IRC:6 Annex B)

The tool holds one factor table and multiplies it out automatically; you only tick which loads apply. Values marked *verify* are left for you to confirm against Annex B before they are locked into the tool.

Load	ULS basic (Table B.2)	SLS rare	SLS frequent	SLS quasi-permanent
Self-weight, SIDL	1.35 (1.0 if relieving)	1.0	1.0	1.0

Load	ULS basic (Table B.2)	SLS rare	SLS frequent	SLS quasi-permanent
Wearing coat	1.75	1.0	1.0	1.0
Earth pressure	1.5	1.0	1.0	1.0
LL surcharge	1.2	*verify*	*verify*	*verify*
Vehicle LL + impact	1.5	1.0	0.75	0
SV load	1.15	1.0	0.75	*verify*
Temperature	0.9 (accompanying)	*verify*	*verify*	*verify*

SV factors: IRC:6 cl. 204.5.4. Other ULS values match typical IRC box-culvert calculations (example).

What each combination is used for:

- ULS basic → bending and shear design.
- SLS rare → stress limits in concrete and steel.
- SLS frequent or quasi-permanent → crack width (which one for RCC: *verify* IRC:112 Table 12.1).
- SLS → base pressure vs SBC.

How the tool builds them: primary load cases (one per load and per live-load position) × the factor table = combination list, written to STAAD with numbered titles like `ULS-03 Box empty, LL max sag`. If you later switch to compression-only springs, combinations must become re-run load cases (STAAD `REPEAT LOAD`) because results no longer add linearly.

> **Important note (16 Oct):** this session, study the combinations in the sample STAAD file and explain them; no implementation yet. Because the springs are compression-only, combinations will be built as primary cases with REPEAT LOAD.

Stage 5 – Run and extract with OpenSTAAD (Python)

Python talks to a running STAAD.Pro through COM, exactly like your VBA did, using the `comtypes` library. Bentley's help shows the connection as `client.GetActiveObject("StaadPro.OpenSTAAD")`, and each method is flagged once with `_FlagAsMethod` before first use (Bentley Python guide). Python and STAAD must be the same bitness (both 64-bit).

Job	OpenSTAAD call	Status
Connect	`GetActiveObject("StaadPro.OpenSTAAD")`	confirmed
New file	`NewSTAADFile`	confirmed
Nodes, beams	`Geometry.CreateNode`, `Geometry.CreateBeam`	confirmed
Section	`Property.CreatePrismaticRectangleProperty`, `Property.AssignBeamProperty`	first: check in local help; second confirmed
Supports	`Support.CreateSupportFixedBut` (release + spring values), `Support.AssignSupportToNode`	first: check signature in local help
Load cases	`Load.CreateNewPrimaryLoad`, `Load.SetLoadActive`, `Load.AddSelfWeightInXYZ`	confirmed
Member loads	`Load.AddMemberUniformForce`, `Load.AddMemberConcForce`, trapezoidal load method	trapezoidal: check name in local help
Combinations	`Load.CreateNewLoadCombination`, `Load.AddLoadAndFactorToCombination`	check in local help
Run	`Command.PerformAnalysis`, `Analyze`, `AreResultsAvailable`	confirmed
Results	`Output.GetMemberEndForces`, `Output.GetIntermediateMemberForcesAtDistance`, `Output.GetSupportReactions`	confirmed

Sources: OpenSTAAD multi-language guide, VIKTOR example, intermediate forces syntax. The full method list is in the OpenSTAAD help that ships with your STAAD install.

Recommended hybrid: Python *writes* the model as a plain STAAD input file (`.std` text: joints, members, springs, loads, combinations), then uses OpenSTAAD only to open, run and read results. A text file is easy to read, compare and fix, and avoids most COM quirks during modelling.

Extraction points (per member, for every combination, then enveloped):

- Top and base slab: at wall faces (hogging), at mid-span (sagging), and at a distance d from wall faces (shear).
- Walls: top, bottom, and the maximum along the height; axial force N comes with each.

Output goes to an Excel sheet laid out like your design sheet, so the hand design in Stage 6 reads straight from it.

Helpful extras: the community `openstaad` Python package wraps many of these calls (PyPI), and Bentley has published an official OpenSTAAD MCP server (GitHub).

Stage 6 – Design (IRC:112-2020)

You design; the tool lays out the forces, runs the standard checks, and flags anything that fails. The box is designed as three separate zones, each with its own bar table.

Zone	Sections checked	Tension face
Top slab	wall faces, mid-span, d from face	outer (top) at corners; inner (bottom) at mid-span
Side walls	top, bottom, maximum along height	outer at corners; inner near mid-height; always with axial force N
Base slab	wall faces, mid-span, d from face	outer (bottom) at corners; inner (top) at mid-span

Check	Combination	IRC:112-2020 reference
Bending	ULS basic	Section 8
Walls: bending with axial force	ULS basic	Section 8 (axial + bending)
Shear, no stirrups (normal for culverts)	ULS basic	10.3.2
Minimum steel: 0.26 fctm/fyk × b × d, not below 0.0013 b d	–	16.5.1.1
Distribution steel	–	Section 16 (slabs)
Stress limits in concrete and steel	SLS rare	12.2
Crack width	SLS	12.3.4, Table 12.1
Cover by exposure class	–	14.3.2, Table 14.2
Base pressure ⩽ SBC	SLS	from spring reactions

Clause numbers are listed so you can confirm each one against your copy of IRC:112 (note Amendment No. 8 of June 2025 in the IRC list). The detailed box-culvert conventions (dispersion, effective width) also sit in IRC:SP:13-2022, the second revision of the small-

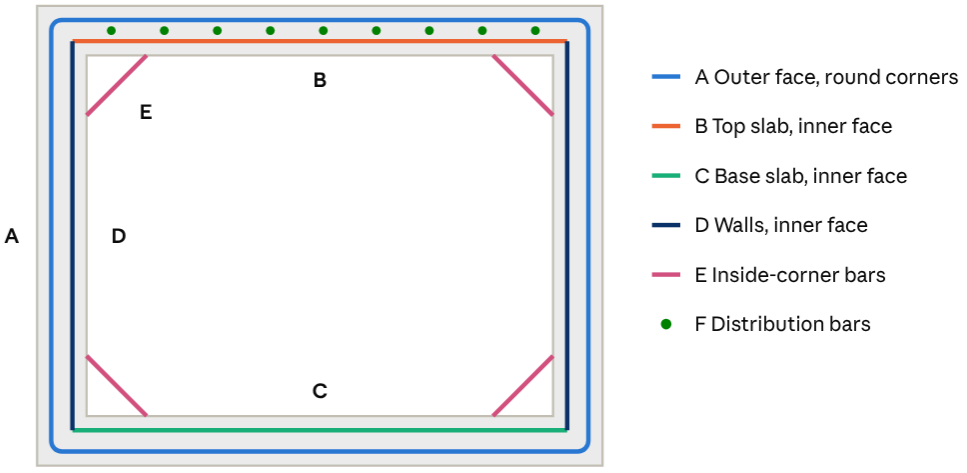
bridges guideline.

Where your judgment comes in: the tool proposes a bar diameter and spacing from your preferred list for each zone and face; you accept or change it. The accepted table is what feeds the drawing.

Stage 7 – Detailing and AutoCAD drafting

The drawing is always the same picture with different numbers, so the tool draws it from your accepted bar table: box size, thicknesses, cover, and for each mark the diameter, spacing and lap.

Bar layout the drawing generator will fill: six bar marks



Marks are placeholders; we will match your drawing standard and bar shapes.

cross-section bar marks · single cell

Outer faces take tension at the corners and inner faces at mid-spans (Stage 6 table), which is why the layout splits into these marks.

Two ways to produce the drawing:

Route	How it works	Needs	Best for
`ezdxf` (start here)	Python writes a .dxf file directly; AutoCAD just opens it	nothing running	batch output, robust, any AutoCAD version opens it
`pyautocad` / `win32com`	Python drives a running AutoCAD through ActiveX/COM	full AutoCAD (LT has no ActiveX)	drawing live inside your company template

A published review of Python-for-AutoCAD libraries found ezdxf the strongest on robustness and documentation (Springer). AutoCAD LT supports AutoLISP from 2024 on Windows, but not the ActiveX calls (autocad-mcp notes).

Drawing job	AutoCAD COM call	ezdxf call
Concrete outline, bars	`AddLine`, `AddLightWeightPolyline`	`add_line`, `add_lwpolyline`
Bar dots (distribution)	`AddCircle`	`add_circle`
Bar mark labels, notes	`AddText`, `AddMText`	`add_text`, `add_mtext`
Dimensions	`AddDimAligned`, `AddDimRotated`	`add_aligned_dim`, `add_linear_dim`
Hatch (concrete)	`AddHatch`	`add_hatch`
Title block, symbols	`InsertBlock`	`add_blockref`
Layers	`Layers.Add`	`layers.add`

COM names come from the AutoCAD ActiveX reference that `pyautocad` wraps (pyautocad docs).

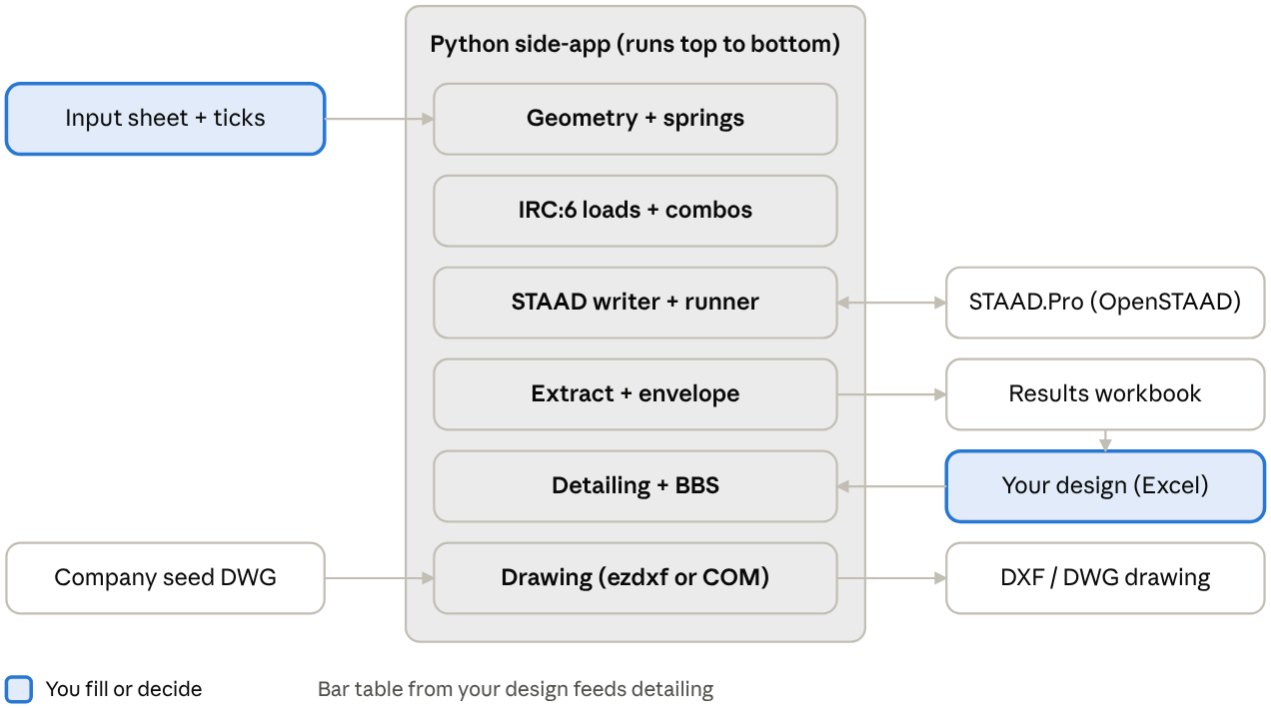
Drawing steps:

1. Open your company seed file (title block, layers, text and dimension styles).
2. Draw the concrete outline and haunches on the concrete layer.
3. Offset by cover and draw each bar mark as a polyline on the rebar layer.
4. Place distribution bars as dots at their spacing.
5. Add bar-mark labels with leaders, then dimensions.
6. Build the bar bending schedule (shape, length, number per metre, weight) as a table.
7. Save as DXF or DWG; plot to PDF for the check.

Tool architecture

A small desktop side-app with one button per stage (Build model, Run, Extract, Draw). STAAD.Pro and AutoCAD stay open beside it; nothing replaces them.

One side-app in the middle; STAAD and AutoCAD stay where they are



tool architecture · 6 modules

Input sheet idea (units are fixed in the template; you only type values and pick Include):

Include	Load	Value	Unit (fixed)
▾	Fill depth over top slab		m
▾	Wearing coat thickness		mm
▾	Soil angle of friction φ		degrees
▾	Vehicle class (70R / Class A / SV)		–
▾	Water inside box (box full)		m depth
▾	Temperature and shrinkage		–

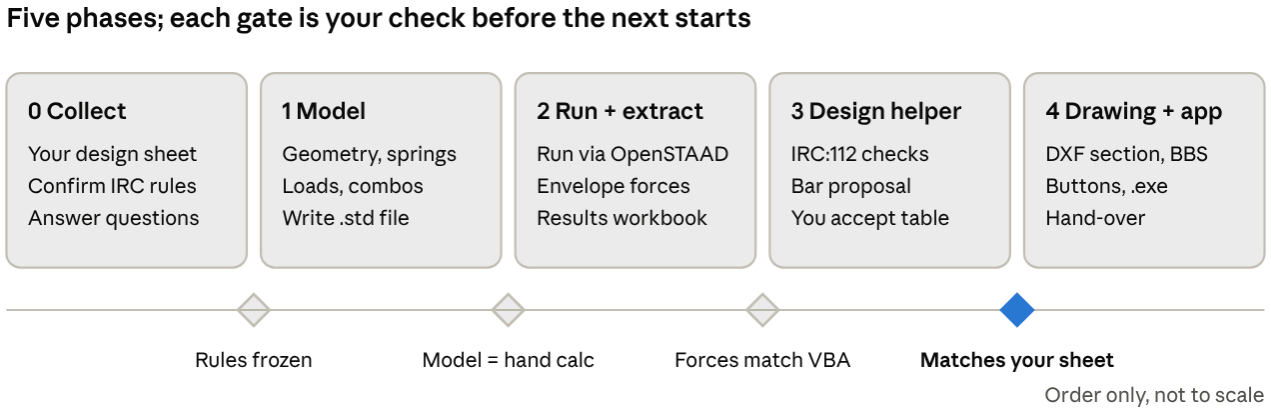
Need	Library	Why
Window with buttons	PySide6 or Tkinter	simple desktop side-app
Read inputs, write results	openpyxl	Excel stays your front end
Talk to STAAD	comtypes	COM access to OpenSTAAD
Write drawings	ezdxf	DXF without AutoCAD running

Need	Library	Why
Drive AutoCAD live (optional)	pyautocad or pywin32	full AutoCAD only
Hand it to colleagues	PyInstaller	one .exe, no Python install

Every run saves a folder: inputs, the .std file, results workbook, drawing. That folder is the audit trail you check against.

Roadmap and verification

Build in five phases and don't start the next until you have passed the gate. The key gate is the fourth: the tool's design must match your own sheet.



roadmap · 5 phases, 4 gates

Double-check tests (tick as each passes):

- [ ] Dead load only: sum of spring reactions equals total self-weight of the box.
- [ ] Symmetric loading gives symmetric moments and equal wall forces.
- [ ] One 4 × 4 box matches your existing STAAD model and VBA extraction.
- [ ] Design output matches your design sheet for a 4 × 4 and a 4 × 5 box.
- [ ] Drawing dimensions, re-measured in AutoCAD, match the inputs.
- [ ] Bar schedule totals match a hand count for one metre strip.

Open questions for you

These decide how the tool is built. Answer them directly here or by comment, and share your design sheet when ready.

Model and springs

- [ ] You mentioned "1.5" for the cross-section. Width is 1.0 m; what does 1.5 refer to?

- [ ] End spring rule: half segment + half wall (geometric), Bowles-style doubling, or your own formula?
- [ ] Which FOS do you normally use in 40 × FOS × SBC?
- [ ] Horizontal restraint at the base: one fixed node or horizontal springs?
- [ ] Linear springs only, or compression-only (lift-off) as well?
- [ ] Haunches at the corners: yes or no, and what size?

Loads and design

- [ ] Live-load dispersion through fill and effective-width rule used in your sheet.
- [ ] Vehicle classes you normally run (70R, Class A, SV?).
- [ ] Do you include temperature and shrinkage for boxes with little or no fill?
- [ ] Exposure class and cover you normally adopt; which SLS combination you use for crack width.
- [ ] Walls: designed for bending only, or bending with axial force?

Detailing and software

- [ ] "Points rebars": did you mean corner (haunch) bars, or something else?
- [ ] Please share one finished drawing and BBS so the generator copies your bar marks, shapes and title block.
- [ ] STAAD.Pro version, and full AutoCAD or AutoCAD LT (LT means the DXF route only).

Sources

- IRC:6-2017 (incl. amendments to Oct 2019)
- IRC amendment list (Amendment No. 9 to IRC:6, March 2026)
- Bowles subgrade modulus summary
- Bentley: OpenSTAAD in Python
- OpenSTAAD multi-language guide
- VIKTOR: Python to STAAD.Pro
- Python-for-AutoCAD library review (Springer)
- pyautocad docs